



DESC



HISTORY



RULES



README



SETTINGS

Codewriting

N sweets are kept before Alice. Sweets are numbered from 0 to $N-1$, and the time Alice takes to eat the i th sweet is $time[i]$ minutes.

Given the array `time` containing the time each sweet needs to be eaten, write a program to find the minimum sum, of the time required to eat each sweet, if only one sweet can be eaten by Alice at a moment.

Constraints

- $1 \leq N \leq 1000$
- $1 \leq time[i] \leq 100$

Sample 0

Input

`time = [2, 1, 4, 5, 1]`

Output

28

Explanation

If you line up the sweets in the order $[S1, S4, S0, S2, S3]$, $S1$ needs 1 minute, $S4$ needs $1+1 = 2$ minutes, $S0$ needs $1+1+2 = 4$ minutes, $S2$ needs $1+1+2+4 = 8$ minutes and $S3$ needs $1+1+2+4+5 = 13$ minutes.

The sum of the time each sweet needs is $1+2+4+8+13 = 28$ minutes. There is no better way other than this to minimize the sum of the total amount of time required.

Sample 1

Python 3

```
1 def
  solution
    (time):
2
3
```

⏮ RUN TESTS

⋮ MORE

0/100

VIEW DIFF

SUBMIT

SKIP



DESC



HISTORY



RULES



README



SETTINGS

Sample 1

Input

```
times = [1]
```

Output

```
1
```

Explanation

S0 is only there and it will take only 1 minute.

- [execution time limit] 4 seconds (py3)
- [memory limit] 1 GB
- [input] array.integer time

Random

- [output] integer

[Python 3] Syntax Tips

```
# Prints help message to the console
# Returns a string
def helloworld(name):
    print("This prints to the console when you Run Tests")
    return "Hello, " + name
```

Python 3

```
1 def
  solution
  (time):
2
3
```

[RUN TESTS](#)[MORE](#)

0/100

[VIEW DIFF](#)[SUBMIT](#)[SKIP](#)



DESC



HISTORY



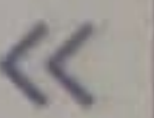
RULES



README



SETTINGS

 Codewriting

Problem

Uber's system is under a DDoS attack, and you are tasked with designing a rate limiter to prevent the system from being overwhelmed. The rate limiter should control the number of requests allowed for the API's within a specific time interval. if the API requests exceeds the defined limit in the interval ,all the excess calls are blocked/denied.

You are given:

- A list of API's for which rate limiter configurations are provided and their corresponding rate limits.
(Each API in the list has a specific rate limit configuration provided in a one-to-one mapping with the API list. i.e, rateLimits[i] is the rate limit config of apiList[i].)
- A default rate limit configuration, should be used incase API specific configuration is not provided.(All APIs not listed in apiList will use the default rate limit configuration.)
- A list of requests along with their timestamps.

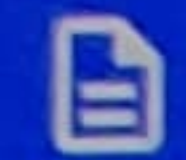
Determine the list of API's which are rate-limited (i.e., denied) and the number of denied for each API .

Input:

- apiList - A list of API's.
- rateLimits - A list of rate limit configurations where each sublist contains two integers: [limit, interval].
[l,i] implies only l requests are allowed in any time window of size i seconds.
rateLimits[i] is the rate limit config of api apiList[i]
- defaultLimits - A list containing two integers: [default_limit, default_interval] representing the default configuration.
- requests - A list of requests where each sublist contains:
A string request_type.
A string representing the timestamp (convert this to integer for processing).

Output:

For each API which are denied, output the total number of requests that were denied due to exceeding the rate limit. return the result in ascending order of API names.



DESC



HISTORY



RULES



README



SETTINGS

Output:

For each API which are denied, output the total number of requests that were denied due to exceeding the rate limit. **return the result in ascending order of API names.**

Example :

input :

- `apiList = ["type1", "type2"]`
- `rateLimits = [[5, 10], [2, 5]]`
- `defaultLimits = [1, 10]`
- `requests = [["type1", "1"], ["type1", "2"], ["type1", "3"], ["type1", "11"], ["type1", "12"], ["type2", "1"], ["type2", "2"], ["type2", "6"], ["type2", "5"], ["type3", "1"], ["type4", "9"], ["type4", "14"]]`

output :

`[["type2", "1"], ["type4", "1"]]`

Explanation:

- type1 has a limit of 5 requests per 10 seconds(i.e, only 5 requests are allowed in any 10 seconds time window). All requests for type1 are within limits.
- type2 has a limit of 2 requests per 5 seconds(i.e, only 2 requests are allowed in any 5 seconds time window). The 3rd type2 request at 5 seconds within the interval [1, 5] is rate-limited.
- type3 uses the default limit of 1 requests per 10 seconds. The only request for type3 is within limits.
- type4 uses the default limit of 1 requests per 10 seconds. The 2nd type4 request at 14 seconds within the interval [9, 19] is ratelimited.

Constraints:

- $0 \leq \text{number of requests} \leq 10^5$
- $1 \leq \text{limits} \leq 10^5$

Python 3

```
1 def  
  solution  
  (apiList  
  ,  
  rateLimi  
  ts,  
  defaultL  
  imits,  
  requests  
  ):  
2  
3
```

RUN TESTS

MORE

0/150

VIEW DIFF

SUBMIT

SKIP



DESC



HISTORY



RULES



README



SETTINGS

Constraints:

- $0 \leq \text{number of requests} \leq 10^5$
- $1 \leq \text{limits} \leq 10^5$
- $1 \leq \text{intervals} \leq 10^5$ (interval in seconds)
- $1 \leq \text{timestamps} \leq 10^9$

• [execution time limit] 4 seconds (py3)

• [memory limit] 1 GB

• [input] array.string apiList

• [input] array.array.integer64 rateLimits

• [input] array.integer64 defaultLimits

• [input] array.array.string requests

• [output] array.array.string

[Python 3] Syntax Tips

```
# Prints help message to the console
# Returns a string
def helloworld(name):
    print("This prints to the console when you Run Tests")
    return "Hello, " + name
```



Python 3

```
1 def
  solution
  (apiList
  ,
  rateLimits,
  defaultLimits,
  requests
  ):
2
3
```

2
3

: MORE

0/150

VIEW DIFF

SUBMIT

SKIP



DESC



HISTORY



RULES



README



SETTINGS

Codewriting

You are on a floor with $n \times n$ rooms laid out in a grid pattern. Each room has some `entryTime[i][j]` before which you cannot enter that room. At current time $t = 0$, you are in the room `(0,0)`. You can only enter another room that is 4-directionally adjacent to the current room if the current time $(t) \geq$ entry time of the adjacent room (x,y) $t \geq \text{entryTime}[x][y]$ else you have to wait till `entryTime[x][y]`. Calculate the minimum time to enter the room `(n-1, n-1)`.

Note: There is no travel time between two rooms.

Constraints

- $1 \leq n \leq 10^2$
- `entryTime[0][0] = 0`
- $0 \leq \text{entryTime}[i][j] \leq 10^5$ where $0 \leq i < n$ AND $0 \leq j < n$ AND $(i, j) \neq (0, 0)$

Example #1

Input:

- $n = 2$
- `entryTime = [[0,1],[2,3]]`

Output:

2

Explanation:

- At time = 0, you are at room `(0,0)`
- At time = 1, you can enter to room `(0,1)` from `(0,0)`
- At time = 2, you can enter to room `(0,1)` or `(1,0)` from `(0,0)`
- At time = 3, you can enter to destination room `(1,1)` from `(0,1)` or `(1,0)`

```
1 def
  solution
    (n,
     entryTim
     e):
```

```
2
3
```

RUN TESTS

MORE

0/200

VIEW DIFF

SUBMIT

SKIP



DESC



HISTORY



RULES



README



SETTINGS

Example #2

Input:

- $n = 3$
- `entryTime = [[0,17,25],[5,25,50],[10,2,3]]`

Output:

10

Explanation:

- One of the possible path = `(0,0) -> (1,0) -> (2,0) -> (2,1) -> (2,2)`

- [execution time limit] 4 seconds (py3)

- [memory limit] 1 GB

- [input] integer n

length and width of ground

- [input] `array.array.integer` `entryTime`

`entryTime[i][j]` represents the entry time to enter room (i,j)

- [output] integer

minimum time to reach room $(n-1, n-1)$

[Python 3] Syntax Tips

Python 3

```
1 def solution
  (n,
  entryTime
  e):
2
3
```

MORE

RUN TESTS

0/200

VIEW DIFF

SUBMIT

SKIP

< BACK TO QUESTIONS

Question 4 of 4 ▾

0:57:09 ?



DESC



HISTORY



RULES



README



SETTINGS

Codewriting

A cab service company is analyzing n number of new cities to understand if cab service zones should be set up across them. All cities are labeled from 1 to n .

They have a list of roads connecting these cities - $edges$ where $edges[i] = [a_i, b_i]$ which indicates that there is a bidirectional road between city a_i and city b_i and both are neighbours. City a_i is the neighbour of city b_i and vice-versa.

The cities chosen may belong to different countries and continents.

The company wants to experiment **Bilateral Approach** which implies if a cab service zone is set up in a city, then there is no need to set up any zone in its neighbouring cities. If there exists a road from a to b , b to c & a to d and a cab service zone is set up in a , there should not be any zones in b and d since they are neighbours of a but c should have a zone. One city can have only one zone.

They want to divide these cities into m groups (1 -indexed) such that each city belongs to only one group and they are able to follow the above approach.

They need your help to find the maximum number of groups (i.e., maximum m) into which the cities can be divided. Return -1 if it is impossible to group the cities with the given conditions.

Note:

1. There is at most one bidirectional road between any pair of cities
2. Each city should be grouped
3. Each city should either have a zone setup or is a neighbour to a city with a zone

Sample Input 1

$n = 6$, $edges = [[1,2],[1,4],[1,5],[2,6],[2,3],[4,6]]$

Sample Output 1

4

Explanation

Python 3

```
1 def solution
  (n,
  edges):
2
3
```

▶ RUN TESTS

: MORE

0/250

VIEW DIFF

SUBMIT

REVIEW YOUR ANSWERS

DESC

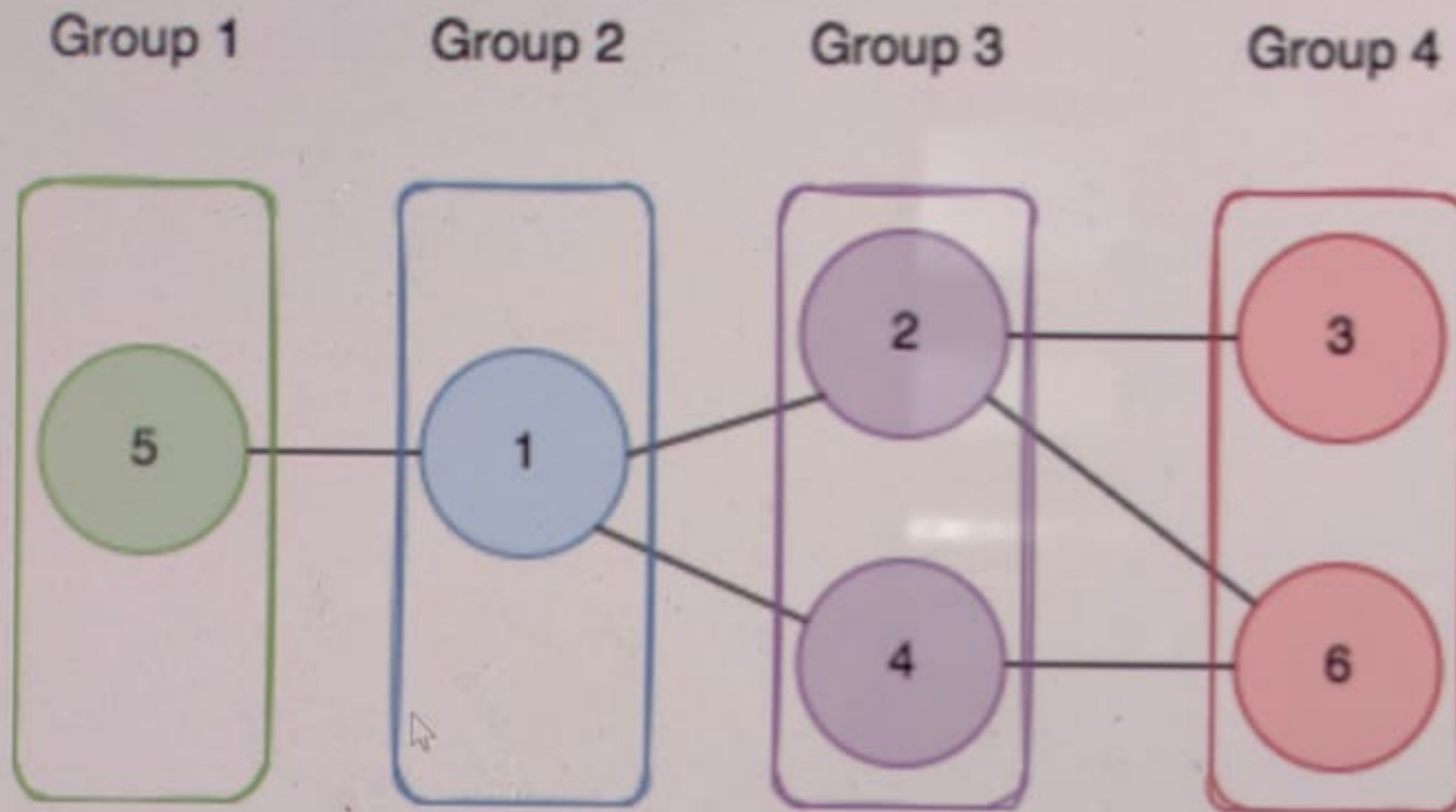
HISTORY

RULES

README

SETTINGS

Explanation



As shown in the diagram,

City 5 is added to Group 1

City 1 is added to Group 2

City 2 and 4 are added to Group 3

City 3 and 6 are added to Group 4

It is possible to follow the *Bilateral Approach* in this scenario and the number of groups are maximum in this case

Sample Input 2

Python 3

```
1 def  
  solution  
  (n,  
   edges):  
2  
3
```

RUN TESTS

MORE

0/250

VIEW DIFF

SUBMIT

REVIEW YOUR ANSWERS

DESC

Sample Input 2

`n = 3, edges = [[1,2],[2,3],[3,1]]`

Sample Output 2

-1

Explanation

Since it is not possible to follow *Bilateral Approach* for above connected cities

Sample Input 3

`n = 4, edges = [[1,2],[1,4]]`

Sample Output 3

4

Explanation

We can setup zones in 2 different ways -

First, zones can be setup in city 1 and 3. Since city 2 and 4 are neighbours are city 1, they do not need zones

Second, zones can be setup in cities 2, 4 and 3. In this case, there should be no zone in city 1 since it is neighbours to both cities 2 and 4

Hence, maximum number of groups can be 4 in both the above cases such that all the cities are in a different group

- [execution time limit] 4 seconds (py3)
- [memory limit] 1 GB
- [input] integer n

n is the number of cities ($1 \leq n \leq 500$)

Python 3

```
1 def
  solution
    (n,
     edges):
2
3
```

 RUN TESTS MORE

0/250

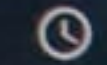
VIEW DIFF

SUBMIT

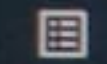
REVIEW YOUR ANSWERS



DESC



HISTORY



RULES



README



SETTINGS

- [execution time limit] 4 seconds (py3)

- [memory limit] 1 GB

- [input] integer n

n is the number of cities ($1 \leq n \leq 500$)

- [input] array.array.integer edges

A 2D array of integers where

$1 \leq \text{edges.length} \leq 10^4$

$\text{edges}[i].\text{length} == 2$

$1 \leq a_i, b_i \leq n$

$a_i \neq b_i$

Each pair of cities is unique. Also if (a, b) exists, then (b, a) does not exists in the array

Each $\text{edges}[i] = \{a_i, b_i\}$ which indicates a_i and b_i are cities and there is a bidirectional road connecting city a_i and city b_i

- [output] integer

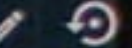
Maximum number of groups into which you can divide the cities. -1 if it is impossible to group the cities.

[Python 3] Syntax Tips

```
# Prints help message to the console
# Returns a string
def helloworld(name):
    print("This prints to the console when you Run Tests")
    return "Hello, " + name
```



Python 3



```
1 def
  solution
    (n,
      edges):
2
3
```

2
3

RUN TESTS

: MORE

0/250

VIEW DIFF

SUBMIT

REVIEW YOUR ANSWERS